

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

Lab 6. Mạng Transformer & Cơ Chế Attention

Hà Minh Tuấn

Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 21 tháng 4 năm 2026

Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu và cài đặt được **Scaled Dot-Product Attention** và **Multi-Head Attention** từ đầu bằng PyTorch.
- Cài đặt **Positional Encoding**, **Feed-Forward Network**, **Encoder Layer** và **Decoder Layer**.
- Ghép các khối để xây dựng một mô hình **Transformer** hoàn chỉnh.
- Sử dụng các mô hình pretrained từ **Hugging Face** để giải quyết bài toán NLP thực tế.
- Phân tích, đánh giá và tinh chỉnh (fine-tune) mô hình cho tập dữ liệu cụ thể.

Công cụ & Môi trường

- **Ngôn ngữ:** Python 3.9+ với PyTorch ≥ 2.0 , Hugging Face **transformers** ≥ 4.30 .
- **Nền tảng:** Google Colab (có GPU T4 miễn phí) hoặc Kaggle Notebook (GPU P100/T4 miễn phí 30h/tuần).
- **Thư viện cần cài:**

```
!pip install torch transformers datasets scikit-learn matplotlib seaborn -q
```

1. PHẦN 1 – Thực hành trên lớp: Cài đặt Transformer từ đầu

Hướng dẫn thực hiện Phần 1

Sinh viên **gõ lại** (không copy-paste) từng đoạn code bên dưới vào Google Colab hoặc Jupyter Notebook, chạy thử và trả lời **các câu hỏi phụ** đi kèm. Sau mỗi mục, viết phần trả lời ngắn gọn (3–5 câu) vào cell Markdown trong notebook.

1.1. Khởi tạo môi trường và thư viện

Bước 0: Import và thiết lập

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import numpy as np

# Đặt seed để kết quả có thể tái lập
torch.manual_seed(42)
np.random.seed(42)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

Câu hỏi 0.1

- (a) Tại sao chúng ta cần gọi `torch.manual_seed(42)`? Nếu bỏ dòng này đi, kết quả chạy hai lần có khác nhau không?
- (b) Lệnh `torch.device('cuda' if ... else 'cpu')` có ý nghĩa gì? Trên Colab, làm thế nào để bật GPU trước khi chạy?

1.2. Scaled Dot-Product Attention

Cơ chế attention cốt lõi của Transformer được định nghĩa bởi công thức:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Bước 1: Cài đặt Scaled Dot-Product Attention

```
class ScaledDotProductAttention(nn.Module):
    """ Scaled Dot-Product Attention """
    def __init__(self, temperature, attn_dropout=0.1):
        super().__init__()
        self.temperature = temperature
        self.dropout = nn.Dropout(attn_dropout)

    def forward(self, q, k, v, mask=None):
        # q, k, v: [batch_size, n_heads, seq_len, d_k or d_v]
        attn = torch.matmul(q / self.temperature, k.transpose(2, 3))

        if mask is not None:
            attn = attn.masked_fill(mask == 0, -1e9)

        attn = self.dropout(F.softmax(attn, dim=-1))
        output = torch.matmul(attn, v)
        return output, attn
```

Bước 1 (tiếp): Chạy thử và quan sát kết quả

```
batch_size, seq_len, d_k, d_v = 2, 5, 16, 20
q = torch.randn(batch_size, 1, seq_len, d_k).to(device)
k = torch.randn(batch_size, 1, seq_len, d_k).to(device)
v = torch.randn(batch_size, 1, seq_len, d_v).to(device)

attention = ScaledDotProductAttention(temperature=np.power(d_k,
    ↪ 0.5)).to(device)
output, attn_weights = attention(q, k, v)

print("Output shape:", output.shape)
print("Attention weights shape:", attn_weights.shape)
print("Tổng trọng số mỗi hàng (phải xấp xỉ 1):", attn_weights[0, 0,
    ↪ 0].sum().item())
```

Câu hỏi 1.1 – Tham số temperature và tác động của d_k

- (a) Với $d_k = 16$, giá trị temperature bằng bao nhiêu? Nếu tăng d_k lên 64, 256, 1024 thì temperature lần lượt bằng bao nhiêu?
- (b) **Phân tích toán học:** Giả sử các phần tử của Q và K là các biến ngẫu nhiên độc lập có trung bình 0, phương sai 1. Chứng minh rằng tích vô hướng $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ có phương sai d_k . Do đó khi d_k lớn, tích vô hướng có độ

lớn $\sim \sqrt{d_k}$ – giải thích tại sao phải chia cho $\sqrt{d_k}$ để giữ ổn định gradient của softmax.

- (c) Điều gì xảy ra với **gradient** khi không chia $\sqrt{d_k}$ và d_k lớn? (Gợi ý: softmax bão hòa $\rightarrow$ vanishing gradient).
- (d) Softmax được áp dụng trên `dim=-1`. Với tensor shape `[batch, n_heads, seq_len, seq_len]`, `dim=-1` tương ứng với chiều nào? Tại sao không dùng `dim=-2`?

Câu hỏi 1.2 – Phân tích shape tensor trong Attention

Với cấu hình `batch_size=2`, `seq_len=5`, `d_k=16`, `d_v=20` (chưa có multi-head, chỉ có 1 head):

- (a) Shape của Q, K, V sau khi `unsqueeze(1)` là gì? Chiều thứ hai bằng 1 đại diện cho điều gì?
- (b) **Tính tay:** Shape của `attn = Q @ K.transpose(2,3)` là gì? Shape của `output = attn @ V` là gì? Ghi rõ từng phép nhân ma trận.
- (c) Nếu `seq_len_q  $\neq$  seq_len_k` (ví dụ trong cross-attention: `seq_len_q = 10`, `seq_len_k = 15`), shape của `attn` và `output` lần lượt là gì?

Câu hỏi 1.3 – Thí nghiệm với temperature

Chạy lại code với 3 giá trị `temperature`: 1.0, $\sqrt{d_k} = 4.0$, và $d_k = 16.0$. In ra `attn_weights[0, 0, 0]` cho từng trường hợp:

- So sánh **độ nhọn** (entropy) của phân phối softmax giữa ba trường hợp.
- Nhận xét: `temperature` nhỏ $\rightarrow$ phân phối tập trung hay phân tán? `temperature` lớn $\rightarrow$?
- Từ đó giải thích vai trò điều hòa (regularization) của $\sqrt{d_k}$.

Câu hỏi 1.4 – Tham số mask và attn_dropout

- (a) Giá trị mask `-1e9` được chọn thay vì `-inf` có lý do gì? Nếu dùng `-inf`, phép nhân với 0 có thể sinh ra giá trị NaN – đúng hay sai?
- (b) `attn_dropout=0.1` được áp dụng **sau softmax**, không phải trước. Nếu hoán đổi thứ tự (dropout trước softmax) thì tổng trọng số attention có còn bằng 1 không? Giải thích.

(c) Tham số `attn_dropout` khác với `dropout` ở các lớp khác (FFN, residual) chỗ nào về mặt ngữ nghĩa? Khi nào ta muốn tăng `attn_dropout`?

1.3. Multi-Head Attention

Multi-Head Attention cho phép mô hình "nhìn" thông tin từ nhiều *không gian con biểu diễn* khác nhau cùng lúc:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

với $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Bước 2: Cài đặt Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, n_head, d_model, d_k, d_v, dropout=0.1):
        super().__init__()
        self.n_head = n_head
        self.d_k = d_k
        self.d_v = d_v

        # Các ma trận chiếu tuyến tính  $W^Q, W^K, W^V, W^O$ 
        self.w_qs = nn.Linear(d_model, n_head * d_k, bias=False)
        self.w_ks = nn.Linear(d_model, n_head * d_k, bias=False)
        self.w_vs = nn.Linear(d_model, n_head * d_v, bias=False)
        self.fc = nn.Linear(n_head * d_v, d_model, bias=False)

        self.attention = ScaledDotProductAttention(temperature=d_k ** 0.5)
        self.dropout = nn.Dropout(dropout)
        self.layer_norm = nn.LayerNorm(d_model, eps=1e-6)

    def forward(self, q, k, v, mask=None):
        d_k, d_v, n_head = self.d_k, self.d_v, self.n_head
        sz_b, len_q, len_k, len_v = q.size(0), q.size(1), k.size(1),
        ↪ v.size(1)

        residual = q # Lưu lại cho residual connection

        # Chiếu và tách thành nhiều head: (b, len, d_model) -> (b, len,
        ↪ n_head, d_k)
        q = self.w_qs(q).view(sz_b, len_q, n_head, d_k)
        k = self.w_ks(k).view(sz_b, len_k, n_head, d_k)
```

```

v = self.w_vs(v).view(sz_b, len_v, n_head, d_v)

# Đổi trục: (b, n_head, len, d_k) để tính attention song song trên
↪ các head
q, k, v = q.transpose(1, 2), k.transpose(1, 2), v.transpose(1, 2)

if mask is not None:
    mask = mask.unsqueeze(1)    # Broadcast qua chiều n_head

output, attn = self.attention(q, k, v, mask=mask)

# Ghép các head lại: (b, n_head, len, d_v) -> (b, len, n_head*d_v)
output = output.transpose(1, 2).contiguous().view(sz_b, len_q, -1)
output = self.dropout(self.fc(output))
output = output + residual      # Residual
output = self.layer_norm(output) # LayerNorm
return output, attn

```

Bước 2 (tiếp): Kiểm thử self-attention

```

batch_size, seq_len, d_model, n_head = 2, 5, 512, 8
d_k = d_v = d_model // n_head    # Đảm bảo d_model = n_head * d_k

x = torch.randn(batch_size, seq_len, d_model).to(device)
mha = MultiHeadAttention(n_head, d_model, d_k, d_v).to(device)

output, attn = mha(x, x, x)    # Self-attention: Q=K=V=x
print("Input shape:", x.shape)
print("Output shape:", output.shape)
print("Attention weights shape:", attn.shape)

```

Câu hỏi 2.1 – Mối quan hệ giữa d_{model} , n_{head} , d_k , d_v

- Với $d_{\text{model}} = 512$, $n_{\text{head}} = 8$, giá trị mặc định của d_k , d_v là bao nhiêu? Công thức chung là gì?
- Ràng buộc** $n_{\text{head}} \times d_v = d_{\text{model}}$: ràng buộc này đảm bảo điều gì trong bước `self.fc(output)` – chiều nào phải khớp với chiều nào?
- Thử thay đổi $n_{\text{head}} = 7$ (không chia hết 512). Chạy code và quan sát lỗi (nếu có). Giải thích nguyên nhân dựa trên phép `.view(sz_b, len_q, n_head, d_k)`.
- Có **bắt buộc** $d_k = d_v$ không? Trong code này tại sao ta hay đặt bằng nhau?

Nếu chọn $d_k = 64$, $d_v = 128$ thì shape các ma trận W^Q, W^K, W^V, W^O sẽ là gì?

Câu hỏi 2.2 – Phân tích shape trong Multi-Head Attention

Với `batch_size=2`, `seq_len=5`, `d_model=512`, `n_head=8`, `d_k=d_v=64`, điền shape vào các bước sau:

Biểu thức	Shape
<code>q</code> (đầu vào)	<code>[__, __, __]</code>
<code>self.w_qs(q)</code>	<code>[__, __, __]</code>
<code>.view(sz_b, len_q, n_head, d_k)</code>	<code>[__, __, __, __]</code>
<code>.transpose(1, 2)</code>	<code>[__, __, __, __]</code>
<code>attn</code> (sau softmax)	<code>[__, __, __, __]</code>
<code>output</code> (sau attention)	<code>[__, __, __, __]</code>
<code>.transpose(1, 2).contiguous().view(...)</code>	<code>[__, __, __]</code>
<code>self.fc(output)</code> (cuối cùng)	<code>[__, __, __]</code>

Sau đó in thực tế bằng `print(q.shape)` ở từng bước và so sánh với kết quả điền tay.

Câu hỏi 2.3 – Đếm tham số từng khối

Với `d_model = 512`, `n_head = 8`, `d_k = d_v = 64`:

- Số tham số của `w_qs = nn.Linear(d_model, n_head * d_k, bias=False)` là bao nhiêu? (Gợi ý: `in_features × out_features`).
- Tương tự cho `w_ks`, `w_vs`, `fc`.
- `layer_norm = nn.LayerNorm(d_model)` có bao nhiêu tham số? (Gợi ý: gồm γ và β , mỗi cái có `d_model` phần tử).
- Tổng cộng:** Tính tổng số tham số của `MultiHeadAttention`. Kiểm tra bằng code:

```
mha = MultiHeadAttention(n_head=8, d_model=512, d_k=64, d_v=64)
print(f"Tổng tham số: {sum(p.numel() for p in
    ↪ mha.parameters()):,}")
# In chi tiết từng module
for name, p in mha.named_parameters():
    print(f" {name}: {p.shape} -> {p.numel():,}")
```

- (e) **So sánh:** Nếu giảm xuống `d_model = 256`, `n_head = 4`, tổng tham số giảm bao nhiêu lần? Rút ra nhận xét về **scaling** của Transformer.

Câu hỏi 2.4 – Self-Attention vs Cross-Attention theo tham số

- (a) Trong **self-attention**: truyền `mha(x, x, x)`. Khi đó `len_q`, `len_k`, `len_v` có quan hệ thế nào?
- (b) Trong **cross-attention** của Decoder: `mha(dec_output, enc_output, enc_output)`. Khi nguồn (encoder) dài 50 tokens, đích (decoder) dài 20 tokens, shape của ma trận attention là gì?
- (c) Khi đổi `q`, `k`, `v` thì shape của **output** phụ thuộc vào `len_q` hay `len_k`? Giải thích tại sao.

Câu hỏi 2.5 – Vai trò của dropout và LayerNorm

- (a) Trong `MultiHeadAttention` có **hai** chỗ dùng dropout: bên trong `ScaledDotProductAttention` (`attn_dropout`) và sau `self.fc` (`dropout`). Giá trị mặc định đều là 0.1. Hai chỗ này áp dụng dropout lên cái gì khác nhau?
- (b) `LayerNorm(d_model, eps=1e-6)`: tại sao chuẩn hóa trên chiều `d_model` mà không phải chiều `seq_len`? **Eps = 1e-6** là gì, tại sao không thể đặt `eps = 0`?
- (c) Sơ đồ gốc của Transformer dùng **Post-LN**: `LayerNorm(x + Sublayer(x))` (giống code trong bài). Các mô hình mới (GPT-2, ViT) dùng **Pre-LN**: `x + Sublayer(LayerNorm(x))`. Hai cách khác nhau thế nào? Pre-LN có lợi gì khi huấn luyện mạng rất sâu?

1.4. Positional Encoding

Vì Transformer không có tính *tuần tự* như RNN, ta cần bổ sung thông tin vị trí. Công thức sinusoidal:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Bước 3: Cài đặt Positional Encoding

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_hid, n_position=200):
        super().__init__()
        self.register_buffer('pos_table',
```



```

        self._get_sinusoid_encoding_table(n_position,
        ↪ d_hid))

def _get_sinusoid_encoding_table(self, n_position, d_hid):
    def get_position_angle_vec(position):
        return [position / np.power(10000, 2 * (hid_j // 2) / d_hid)
                for hid_j in range(d_hid)]
    sinusoid_table = np.array([get_position_angle_vec(pos_i)
                              for pos_i in range(n_position)])
    sinusoid_table[:, 0::2] = np.sin(sinusoid_table[:, 0::2]) # dim
    ↪ chẵn
    sinusoid_table[:, 1::2] = np.cos(sinusoid_table[:, 1::2]) # dim lẻ
    return torch.FloatTensor(sinusoid_table).unsqueeze(0)

def forward(self, x):
    # x: [batch_size, seq_len, d_model]
    return x + self.pos_table[:, :x.size(1)].clone().detach()

```

Bước 3 (tiếp): Trực quan hóa Positional Encoding

```

import matplotlib.pyplot as plt

d_model_vis, n_pos_vis = 64, 100
pe = PositionalEncoding(d_model_vis, n_position=n_pos_vis)
pe_table = pe.pos_table.squeeze(0).numpy() # [n_position, d_model]

plt.figure(figsize=(10, 5))
plt.imshow(pe_table, aspect='auto', cmap='RdBu_r')
plt.xlabel('Chiều embedding (d_model)')
plt.ylabel('Vị trí (position)')
plt.title('Positional Encoding (sinusoidal)')
plt.colorbar()
plt.show()

```

Câu hỏi 3.1 – Phân tích các tham số của Positional Encoding

- d_{hid} trong PositionalEncoding thường phải bằng d_{model} . Tại sao? (Gợi ý: phép cộng $x + pos_table$ yêu cầu điều gì về shape).
- $n_position=200$ quyết định điều gì? Nếu câu đầu vào dài 300 tokens, chạy code sẽ bị lỗi ở đâu? Thử và quan sát.
- Số 10000 trong công thức $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$ có vai trò gì?

Thử thay bằng 100 và 100000, vẽ lại heatmap, so sánh. (Gợi ý: liên quan đến *bước sóng* của các tần số).

- (d) Shape của `pos_table` là `[1, n_position, d_hid]` (có `unsqueeze(0)`). Tại sao cần chiều batch bằng 1 ở đầu? (Gợi ý: *broadcasting*).

Câu hỏi 3.2 – Register buffer và lưu trữ tham số

- (a) `self.register_buffer('pos_table', ...)` khác với `self.pos_table = nn.Parameter(...)` ở điểm nào về:

- Có được cập nhật bởi `optimizer.step()` không?
- Có xuất hiện trong `model.parameters()` không?
- Có được lưu khi gọi `torch.save(model.state_dict())` không?
- Có tự động chuyển sang GPU khi gọi `model.to('cuda')` không?

- (b) Kiểm tra bằng code:

- (c) **Thí nghiệm:** Nếu ta muốn **học** positional encoding (thay vì cố định sinusoidal), phải thay đổi gì trong code? Đây chính là cách BERT làm – gọi là **learnable positional embedding**.

```
pe = PositionalEncoding(d_hid=64, n_position=100)
print("Có trong parameters():", 'pos_table' in dict(pe.named_parameters()))
print("Có trong buffers():", 'pos_table' in dict(pe.named_buffers()))
print("Có trong state_dict():", 'pos_table' in pe.state_dict())
```

Câu hỏi 3.3 – Quan sát heatmap và ngoại suy

Vẽ heatmap với `d_model_vis = 64` và `n_pos_vis = 100`. Trả lời:

- (a) Ở các chiều **nhỏ** ($i = 0, 1, 2$), giá trị thay đổi nhanh hay chậm theo vị trí? Ở các chiều **lớn** ($i \approx d_{\text{model}}/2$), thì thế nào? Giải thích bằng công thức bước sóng $\lambda_i = 2\pi \cdot 10000^{2i/d_{\text{model}}}$.
- (b) **Tính chất quan trọng:** Chứng minh (hoặc kiểm tra bằng code) rằng PE_{pos+k} có thể biểu diễn dưới dạng **tổ hợp tuyến tính** của PE_{pos} (độc lập với pos). Đây là lý do Transformer có thể "hiểu" được khoảng cách tương đối.
- (c) Nếu huấn luyện với câu dài tối đa 100 tokens, lúc inference gặp câu dài 150 tokens thì: (i) sinusoidal encoding vẫn hoạt động hay không? (ii) learnable embedding của BERT thì sao?

1.5. Position-wise Feed-Forward Network

Bước 4: Cài đặt FFN

```
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_in, d_hid, dropout=0.1):
        super().__init__()
        self.w_1 = nn.Linear(d_in, d_hid)
        self.w_2 = nn.Linear(d_hid, d_in)
        self.layer_norm = nn.LayerNorm(d_in, eps=1e-6)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        residual = x
        x = self.w_2(F.relu(self.w_1(x)))    #  $FFN(x) = W_2 * ReLU(W_1 * x)$ 
        x = self.dropout(x)
        x = x + residual
        x = self.layer_norm(x)
        return x
```

Câu hỏi 4.1 – Tham số d_{in} và d_{hid} của FFN

- (a) Trong code, $w_1 = nn.Linear(d_{in}, d_{hid})$ và $w_2 = nn.Linear(d_{hid}, d_{in})$. Tại sao w_2 phải đưa kích thước về d_{in} ? (Gợi ý: residual $x + residual$).
- (b) Tỷ lệ $d_{hid} / d_{in} = 4$ là quy ước trong paper gốc ($512 \rightarrow 2048$). Mục đích của việc **giãn rồi nén** là gì? (Gợi ý: không gian biểu diễn trung gian rộng hơn $\rightarrow$ học được nhiều *feature interactions* hơn).
- (c) **Tính số tham số:** Với $d_{in} = 512$, $d_{hid} = 2048$, số tham số của w_1 và w_2 lần lượt là bao nhiêu (kể cả bias)? Tổng cộng PositionwiseFeedForward có bao nhiêu tham số?
- (d) **So sánh với MultiHeadAttention:** Với cùng $d_{model} = 512$, FFN ($d_{hid}=2048$) có bao nhiêu tham số vs. MHA ($n_{head}=8$)? Khối nào chiếm nhiều tham số hơn? Rút ra nhận xét.

Câu hỏi 4.2 – Ngữ nghĩa "position-wise"

- (a) Tại sao gọi là "position-wise"? Cùng một cặp (W_1, W_2) được áp dụng lên

mỗi vị trí của chuỗi **độc lập**. Viết lại công thức:

$$\text{FFN}(x_t) = W_2 \cdot \text{ReLU}(W_1 \cdot x_t + b_1) + b_2, \quad \forall t = 1, \dots, \text{seq_len}$$

Điều này có nghĩa là FFN **không** trộn thông tin giữa các vị trí – đúng hay sai? Việc trộn thông tin xảy ra ở đâu trong Transformer?

- (b) So sánh với `nn.Conv1d(d_in, d_hid, kernel_size=1)`: hai thao tác này có tương đương về mặt toán học không? Thử kiểm tra với một tensor mẫu.

Câu hỏi 4.3 – Tham số dropout trong FFN

- (a) dropout được áp dụng ở đâu trong forward pass? (Sau `w_2`, trước residual).
- (b) Nếu đặt `dropout = 0.5` (quá lớn) cho Transformer, hiện tượng gì có thể xảy ra? Ngược lại `dropout = 0.0`?
- (c) Paper gốc dùng `dropout = 0.1`. BERT cũng dùng `0.1`. Tuy nhiên khi fine-tune cho dataset **nhỏ**, có khi cần tăng lên `0.2-0.3`. Tại sao?

1.6. Encoder Layer và Decoder Layer

Bước 5: Encoder Layer

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, d_inner, n_head, d_k, d_v, dropout=0.1):
        super().__init__()
        self.slf_attn = MultiHeadAttention(n_head, d_model, d_k, d_v,
            ↪ dropout=dropout)
        self.pos_ffn = PositionwiseFeedForward(d_model, d_inner,
            ↪ dropout=dropout)

    def forward(self, enc_input, slf_attn_mask=None):
        enc_output, enc_slf_attn = self.slf_attn(
            enc_input, enc_input, enc_input, mask=slf_attn_mask)
        enc_output = self.pos_ffn(enc_output)
        return enc_output, enc_slf_attn
```

Bước 5 (tiếp): Decoder Layer

```
class DecoderLayer(nn.Module):
    def __init__(self, d_model, d_inner, n_head, d_k, d_v, dropout=0.1):
        super().__init__()
```

```

self.slf_attn = MultiHeadAttention(n_head, d_model, d_k, d_v,
    ↪ dropout=dropout)
self.enc_attn = MultiHeadAttention(n_head, d_model, d_k, d_v,
    ↪ dropout=dropout)
self.pos_ffn = PositionwiseFeedForward(d_model, d_inner,
    ↪ dropout=dropout)

def forward(self, dec_input, enc_output,
            slf_attn_mask=None, dec_enc_attn_mask=None):
    # 1) Masked self-attention trên target
    dec_output, dec_slf_attn = self.slf_attn(
        dec_input, dec_input, dec_input, mask=slf_attn_mask)
    # 2) Cross-attention: Q từ decoder, K, V từ encoder
    dec_output, dec_enc_attn = self.enc_attn(
        dec_output, enc_output, enc_output, mask=dec_enc_attn_mask)
    # 3) FFN
    dec_output = self.pos_ffn(dec_output)
    return dec_output, dec_slf_attn, dec_enc_attn

```

Câu hỏi 5.1 – Tham số của Encoder/Decoder Layer

Hàm khởi tạo `EncoderLayer(d_model, d_inner, n_head, d_k, d_v, dropout=0.1)` có 5 tham số chính.

- Ý nghĩa của `d_inner`? Tại sao tham số này không xuất hiện trong `MultiHeadAttention`?
- Với `d_model = 512`, `d_inner = 2048`, `n_head = 8`, `d_k = d_v = 64`, tính tổng số tham số **một** Encoder Layer (MHA + FFN + 2 LayerNorm).
- DecoderLayer** có thêm `enc_attn`: bao nhiêu tham số thêm vào so với EncoderLayer? Tỷ lệ giữa DecoderLayer và EncoderLayer về số tham số là bao nhiêu?

Câu hỏi 5.2 – Luồng tensor qua một Encoder Layer

Viết shape tensor ở từng bước trong `EncoderLayer.forward()` với input shape `[batch=2, seq_len=10, d_model=512]`:

Biến	Shape
<code>enc_input</code>	<code>[__, __, __]</code>
<code>enc_output</code> sau <code>self.slf_attn(...)</code>	<code>[__, __, __]</code>
<code>enc_slf_attn</code> (attention weights)	<code>[__, __, __, __]</code>
<code>enc_output</code> sau <code>self.pos_ffn(...)</code>	<code>[__, __, __]</code>

Nhận xét: shape của `enc_output` có thay đổi khi đi qua `EncoderLayer` không? Vì sao điều này **quan trọng** đối với việc stack nhiều layer?

Câu hỏi 5.3 – Ba loại Attention trong Transformer

Trong `DecoderLayer` có **hai** MHA, trong `EncoderLayer` có **một** MHA. Tổng cộng Transformer có **ba** loại attention:

Tên	Vị trí	Q, K, V lấy từ đâu	Mask
Encoder Self-Attn	EncoderLayer	_____	_____
Masked Self-Attn	DecoderLayer	_____	_____
Cross-Attn	DecoderLayer	_____	_____

Điền vào bảng trên. Trong Cross-Attention, nếu encoder xử lý câu nguồn dài $L_s = 50$ và decoder đang tạo câu đích dài $L_t = 20$:

- Shape của `attn` (ma trận attention weights) là gì?
- Mỗi hàng của ma trận attention đại diện cho **gì**? (Mỗi token đích "nhìn" vào các token nguồn nào).
- Mask `dec_enc_attn_mask` được lấy từ `src_mask` – tại sao? (Gợi ý: token đích được phép nhìn token padding của nguồn không?).

Câu hỏi 5.4 – Thứ tự các phép biến đổi

Trong `EncoderLayer.forward()`, thứ tự là: `slf_attn` $\rightarrow$ `pos_ffn`. Trong `DecoderLayer.forward()`, thứ tự là: `slf_attn` $\rightarrow$ `enc_attn` $\rightarrow$ `pos_ffn`.

- Tại sao** Masked Self-Attention phải đặt **trước** Cross-Attention trong Decoder, không phải ngược lại? (Gợi ý: ý nghĩa – decoder trước hết "suy nghĩ" về những gì mình đã sinh ra, sau đó mới "tham chiếu" encoder).
- Nếu đổi thứ tự thành: `enc_attn` $\rightarrow$ `slf_attn` $\rightarrow$ `pos_ffn`, kiến trúc vẫn chạy được (về mặt code) nhưng có điểm gì bất hợp lý?

1.7. Ghép thành Transformer hoàn chỉnh

Bước 6: Mô hình Transformer

```
class Transformer(nn.Module):
    def __init__(self, n_src_vocab, n_tgt_vocab, src_pad_idx, tgt_pad_idx,
                  d_word_vec=512, d_model=512, d_inner=2048,
                  n_layers=6, n_head=8, d_k=64, d_v=64,
                  dropout=0.1, n_position=200):
        super().__init__()
        self.src_word_emb = nn.Embedding(n_src_vocab, d_word_vec,
                                          ↪ padding_idx=src_pad_idx)
        self.tgt_word_emb = nn.Embedding(n_tgt_vocab, d_word_vec,
                                          ↪ padding_idx=tgt_pad_idx)
        self.position_enc = PositionalEncoding(d_model,
                                                ↪ n_position=n_position)
        self.dropout = nn.Dropout(p=dropout)

        self.encoder = nn.ModuleList([
            EncoderLayer(d_model, d_inner, n_head, d_k, d_v,
                        ↪ dropout=dropout)
            for _ in range(n_layers)])

        self.decoder = nn.ModuleList([
            DecoderLayer(d_model, d_inner, n_head, d_k, d_v,
                        ↪ dropout=dropout)
            for _ in range(n_layers)])

        self.tgt_word_prj = nn.Linear(d_model, n_tgt_vocab, bias=False)
        self.tgt_word_prj.weight = self.tgt_word_emb.weight # Weight tying
        nn.init.xavier_normal_(self.tgt_word_prj.weight)
        self.d_model = d_model

    def get_pad_mask(self, seq, pad_idx):
        return (seq != pad_idx).unsqueeze(-2)

    def get_subsequent_mask(self, seq):
        sz_b, len_s = seq.size()
        return (1 - torch.triu(torch.ones((1, len_s, len_s),
                                          device=seq.device), diagonal=1)).bool()

    def forward(self, src_seq, tgt_seq):
```

```

src_mask = self.get_pad_mask(src_seq, self.src_word_emb.padding_idx)
tgt_mask = self.get_pad_mask(tgt_seq, self.tgt_word_emb.padding_idx)
↪ \
    & self.get_subsequent_mask(tgt_seq)

# Encoder
enc_output =
↪ self.dropout(self.position_enc(self.src_word_emb(src_seq)))
for enc_layer in self.encoder:
    enc_output, _ = enc_layer(enc_output, slf_attn_mask=src_mask)

# Decoder
dec_output =
↪ self.dropout(self.position_enc(self.tgt_word_emb(tgt_seq)))
for dec_layer in self.decoder:
    dec_output, _, _ = dec_layer(dec_output, enc_output,
                                  slf_attn_mask=tgt_mask,
                                  dec_enc_attn_mask=src_mask)

seq_logit = self.tgt_word_prj(dec_output)
return seq_logit.view(-1, seq_logit.size(2))

```

Bước 6 (tiếp): Chạy thử với dữ liệu ngẫu nhiên

```

src_vocab_size, tgt_vocab_size = 1000, 1200
src_pad_idx = tgt_pad_idx = 0
batch_size, src_len, tgt_len = 2, 10, 12

model = Transformer(
    n_src_vocab=src_vocab_size, n_tgt_vocab=tgt_vocab_size,
    src_pad_idx=src_pad_idx, tgt_pad_idx=tgt_pad_idx,
    n_layers=2    # Giảm số layer cho nhẹ
).to(device)

src_seq = torch.randint(1, src_vocab_size, (batch_size, src_len)).to(device)
tgt_seq = torch.randint(1, tgt_vocab_size, (batch_size, tgt_len)).to(device)
src_seq[0, -2:] = src_pad_idx    # Thêm padding
tgt_seq[0, -3:] = tgt_pad_idx

logits = model(src_seq, tgt_seq)
print("Logits shape:", logits.shape)
print("Total parameters:", sum(p.numel() for p in model.parameters()))

```


Câu hỏi 6.1 – Các siêu tham số của Transformer đầy đủ

Hàm `__init__` của lớp `Transformer` có nhiều tham số. Trả lời ngắn gọn ý nghĩa của từng tham số:

Tham số	Ý nghĩa / giá trị paper gốc
<code>n_src_vocab, n_tgt_vocab</code>	_____
<code>src_pad_idx, tgt_pad_idx</code>	_____
<code>d_word_vec</code>	_____ (paper: _____)
<code>d_model</code>	_____ (paper: _____)
<code>d_inner</code>	_____ (paper: _____)
<code>n_layers</code>	_____ (paper: _____)
<code>n_head, d_k, d_v</code>	_____ (paper: _____)
<code>dropout</code>	_____ (paper: _____)
<code>n_position</code>	_____

Ràng buộc ẩn: Tại sao `d_word_vec = d_model`? Điều gì xảy ra nếu hai giá trị này khác nhau?

Câu hỏi 6.2 – Đếm tham số mô hình Transformer

Với cấu hình **paper gốc** (base model): `d_model=512, d_inner=2048, n_layers=6, n_head=8, d_k=d_v=64`, giả sử `n_src_vocab = n_tgt_vocab = 30000`:

- (a) Số tham số của `src_word_emb` và `tgt_word_emb` lần lượt là bao nhiêu?
- (b) Số tham số **một** `EncoderLayer` (từ Câu hỏi 5.1) và **một** `DecoderLayer` là bao nhiêu?
- (c) Tổng tham số của **6 Encoder Layers + 6 Decoder Layers** là bao nhiêu?
- (d) **Tổng cộng** mô hình có bao nhiêu tham số? Kiểm tra bằng code:

```
model = Transformer(n_src_vocab=30000, n_tgt_vocab=30000,
                    src_pad_idx=0, tgt_pad_idx=0)
total = sum(p.numel() for p in model.parameters())
print(f"Total: {total/1e6:.1f}M tham số")
# So sánh với paper: ~65M cho base, ~213M cho big
```

- (e) **Câu hỏi mở:** Nếu giảm `n_layers` từ 6 xuống 2 (như code trong bài để chạy thử), số tham số giảm bao nhiêu %? Tính chính xác.

Câu hỏi 6.3 – Tham số padding_idx trong Embedding

- (a) `nn.Embedding(n_src_vocab, d_word_vec, padding_idx=src_pad_idx)`: tham số `padding_idx` có tác dụng gì? Gradient của row ứng với `padding_idx` có được cập nhật không?

```
emb = nn.Embedding(10, 4, padding_idx=0)
print("Row 0 ban đầu:", emb.weight[0])
# Sau 1 bước backward, row 0 vẫn là 0
```

- (b) Kiểm tra bằng code:
- (c) Tại sao quy ước đặt `pad_idx = 0`? Nếu đặt `pad_idx = 1` thì có ảnh hưởng gì không?

Câu hỏi 6.4 – Hai loại Mask

Có **hai loại mask** được dùng trong `Transformer.forward()`:

- (a) `get_pad_mask(seq, pad_idx)` trả về tensor shape gì với `seq shape [batch, seq_len]`? Giá trị True/False mang ý nghĩa gì?
- (b) `get_subsequent_mask(seq)` tạo ma trận tam giác dưới. Với `seq_len = 4`, viết ra ma trận 4×4 cụ thể (1 = được nhìn, 0 = che).
- (c) `tgt_mask = pad_mask & subsequent_mask`: tại sao phải là **giao** (&), không phải **hợp** (|)? Cho ví dụ với câu target [w1, w2, w3, PAD] (`pad_idx=0`) và `seq_len=4`, vẽ ma trận mask cuối cùng.
- (d) Encoder chỉ cần `src_mask` (pad mask), **không** cần subsequent mask. Tại sao? (Gợi ý: encoder xử lý **song song**, decoder xử lý **tuần tự**).

Câu hỏi 6.5 – Weight Tying

Dòng `self.tgt_word_prj.weight = self.tgt_word_emb.weight` được gọi là **weight tying**.

- (a) Shape của `tgt_word_emb.weight` là `[n_tgt_vocab, d_model]`. Shape của `tgt_word_prj` (`nn.Linear(d_model, n_tgt_vocab)`) yêu cầu weight có shape gì? Hai shape này có khớp để chia sẻ không? (Gợi ý: PyTorch `nn.Linear` lưu weight dạng `[out_features, in_features]`).
- (b) **Số tham số tiết kiệm được**: Với `n_tgt_vocab = 30000`, `d_model = 512`, weight tying tiết kiệm bao nhiêu tham số?
- (c) Ngoài giảm số tham số, weight tying còn có lợi ích lý thuyết nào? (Gợi ý: liên hệ giữa *không gian embedding* và *phân phối xác suất output*).

Câu hỏi 6.6 – Cấu hình Transformer theo ứng dụng

Dựa trên các tham số đã học, hãy đề xuất cấu hình hợp lý cho các trường hợp sau (và giải thích lý do):

Ứng dụng	d_model	n_head	n_layers	d_inner
Dịch máy (paper gốc)	512	8	6	2048
BERT-base	—	—	—	—
GPT-2 small	—	—	—	—
Mô hình nhỏ cho Colab (học tập)	128	4	2	256

Tra cứu các giá trị cho BERT-base, GPT-2 small, và điền vào bảng.

1.8. Sử dụng mô hình pretrained từ Hugging Face

Bước 7: BERT làm feature extractor

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert = BertModel.from_pretrained('bert-base-uncased').to(device)
bert.eval()

text = "Transformers have revolutionized NLP."
inputs = tokenizer(text, return_tensors='pt').to(device)

print("Tokens:", tokenizer.tokenize(text))
print("Input IDs:", inputs['input_ids'])
print("Attention mask:", inputs['attention_mask'])

with torch.no_grad():
    outputs = bert(**inputs)

print("Last hidden states:", outputs.last_hidden_state.shape)    # (1,
    ↪ seq_len, 768)
print("Pooler output:", outputs.pooler_output.shape)             # (1, 768)
```

Câu hỏi 7.1 – So sánh tham số BERT với Transformer gốc

In thông số BERT-base bằng code:

Trả lời:

- (a) Đối chiếu từng tham số với paper Transformer gốc: BERT-base giống/khác ở

điểm nào về `d_model`, `n_layers`, `n_head`, `d_inner`?

- (b) BERT-base có tổng bao nhiêu tham số? Kiểm tra:

```
print(f"{sum(p.numel() for p in bert.parameters())/1e6:.1f}M")
```

- (c) `max_position_embeddings = 512` trong BERT có ý nghĩa gì? Tại sao BERT **không** xử lý được câu dài hơn 512 tokens (khác với sinusoidal encoding)?
- (d) BERT-base chỉ có **Encoder** (12 layer), không có Decoder. Mô hình nào có cả Encoder và Decoder như Transformer gốc? (T5, BART, mBART, ...)

```
from transformers import BertConfig
cfg = BertConfig.from_pretrained('bert-base-uncased')
print(f"hidden_size (d_model)      = {cfg.hidden_size}")
print(f"num_hidden_layers         = {cfg.num_hidden_layers}")
print(f"num_attention_heads         = {cfg.num_attention_heads}")
print(f"intermediate_size (d_inner) = {cfg.intermediate_size}")
print(f"max_position_embeddings      = {cfg.max_position_embeddings}")
print(f"vocab_size                    = {cfg.vocab_size}")
print(f"hidden_dropout_prob          = {cfg.hidden_dropout_prob}")
```

Câu hỏi 7.2 – Tokenizer và shape input

Với `text = "Transformers have revolutionized NLP."` và BERT tokenizer:

- (a) Quan sát output `tokenizer.tokenize(text)`. Tại sao "revolutionized" bị tách thành nhiều subwords? Thuật toán nào BERT dùng (WordPiece, BPE, SentencePiece)?
- (b) `input_ids` có shape là gì với 1 câu? Hai token **đặc biệt** [CLS] (id=101) và [SEP] (id=102) xuất hiện ở đâu trong chuỗi?
- (c) `attention_mask` có vai trò gì? Giá trị 0 và 1 đại diện cho gì? (Gợi ý: liên hệ với `pad_mask` đã cài đặt ở Câu hỏi 6.4).
- (d) Nếu tokenize một **batch** gồm 3 câu dài ngắn khác nhau, tokenizer sẽ làm gì để chúng có cùng `seq_len`? Thử với:

```
texts = ["Hello world.", "This is a longer sentence.", "Hi."]
batch = tokenizer(texts, padding=True, return_tensors='pt')
print(batch['input_ids'].shape, batch['attention_mask'])
```

Câu hỏi 7.3 – Output của BERT và cách sử dụng

- (a) `last_hidden_state.shape = [batch, seq_len, 768]`. Số 768 từ đâu ra? (Đối chiếu với `cfg.hidden_size`).
- (b) `pooler_output.shape = [batch, 768]`. Đây là biểu diễn của **token** nào? Được tính bằng phép biến đổi gì lên `last_hidden_state`?
- (c) **Lựa chọn cho downstream task:**
- Phân loại câu (sentiment) → nên dùng embedding nào?
 - Phân loại token (NER, POS) → nên dùng embedding nào?
 - Đo độ tương đồng câu (sentence similarity) → dùng [CLS] hay trung bình tất cả token?

Câu hỏi tổng hợp Phần 1 – Bảng tổng hợp tham số Transformer

Hãy điền vào bảng tổng hợp dưới đây để hệ thống hóa kiến thức. Với mỗi tham số, nêu (i) ký hiệu toán học thông dụng, (ii) vai trò, (iii) giá trị paper gốc, (iv) tác động khi tăng/giảm:

Tham số	Ký hiệu	Vai trò	Paper gốc	Tăng → ?
<code>d_model</code>	d_{model}	chiều biểu diễn	512	_____
<code>n_layers</code>	N	_____	6	_____
<code>n_head</code>	h	_____	8	_____
<code>d_k = d_v</code>	d_k	_____	64	_____
<code>d_inner (d_ff)</code>	d_{ff}	_____	2048	_____
<code>dropout</code>	p	_____	0.1	_____
<code>n_position</code>	L_{max}	_____	512	_____
<code>vocab_size</code>	$ V $	_____	~30K	_____

Bài tập tổng hợp: Vẽ sơ đồ khối kiến trúc **Transformer đầy đủ** với shape tensor rõ ràng ở từng bước. Ít nhất phải chỉ rõ:

- Shape của input $X_{\text{src}}, X_{\text{tgt}}$ (sau khi tokenize).
- Shape sau Embedding + Positional Encoding.
- Shape qua **một** Encoder Layer và sau **N** Encoder Layers.
- Shape qua **một** Decoder Layer và sau **N** Decoder Layers.
- Shape output cuối cùng **logits** (trước softmax).
- Vị trí của **residual connection, LayerNorm, Dropout** trong từng sub-layer.

2. PHẦN 2 – Bài tập về nhà: Áp dụng Transformer vào dữ liệu thực tế

Hướng dẫn Phần 2

Sinh viên **chỉ chọn 1 (một) bài toán** trong 5 gợi ý dưới đây để làm. Mỗi bài toán đã được thiết kế để chạy được trên **Google Colab GPU T4 (miễn phí)** hoặc **Kaggle Notebook** trong vòng 30–90 phút.

Yêu cầu nộp:

1. File notebook `.ipynb` có đầy đủ code, kết quả chạy, chú thích tiếng Việt.
2. Báo cáo ngắn (PDF, 3–5 trang) mô tả: (i) bài toán, (ii) cách tiền xử lý dữ liệu, (iii) kiến trúc mô hình, (iv) kết quả đánh giá (accuracy, F1, v.v.), (v) nhận xét và hướng cải tiến.
3. **Chỉ sử dụng tập con** (subset) nếu dataset quá lớn – ưu tiên demo được pipeline hoàn chỉnh, không cần train đạt SOTA.

Gợi ý chung: tất cả 5 bài toán đều sử dụng **dữ liệu tiếng Việt**. Ưu tiên các mô hình đã pretrained cho tiếng Việt: `vinai/phobert-base` (text classification, NER), `nguyenvulebinh/vi-mrc-base` (QA), `VietAI/vit5-base-vietnews-summarization` (summarization), `vinai/bartpho-syllable` (seq2seq). Nếu không tiện tách từ, có thể dùng `xlm-roberta-base` (đa ngôn ngữ). Huấn luyện 2–3 epochs là đủ với `batch_size = 16` hoặc 32.

2.1. Bài toán 1: Phân loại cảm xúc bình luận tiếng Việt (Sentiment Analysis)

Thông tin dữ liệu – các nguồn đã kiểm chứng còn hoạt động

Lựa chọn 1 (khuyến nghị) – UIT-VSFC trên Hugging Face:

https://huggingface.co/datasets/uitnlp/vietnamese_students_feedback

- 16.175 câu phản hồi sinh viên, đã **chia sẵn** train/validation/test (11.426 / 1.583 / 3.166).
- Mỗi câu có **2 nhãn**: **sentiment** (0: negative, 1: neutral, 2: positive) và **topic** (0: lecturer, 1: training_program, 2: facility, 3: others).
- **Tải bằng 1 dòng lệnh**: `load_dataset("uitnlp/vietnamese_students_feedback")`

Lựa chọn 2 (thay thế) – UIT-VSFC mirror:

<https://huggingface.co/datasets/tridm/UIT-VSFC> hoặc <https://huggingface.co/datasets/ura-hcmut/UIT-VSFC>

Lựa chọn 3 (nâng cao) – Vietnamese Comment Sentiment:

<https://huggingface.co/datasets/minhtoan/vietnamese-comment-sentiment>

- Bình luận tiếng Việt từ nhiều mạng xã hội, nhãn positive/negative/neutral.
- Dataset phong phú hơn về văn phong (có emoji, tiếng lóng, viết tắt).

Lựa chọn 4 (nâng cao, aspect-based) – UIT-ViSFD:

<https://huggingface.co/datasets/visolex/ViSFD>

- 11.122 bình luận về điện thoại thông minh từ trang thương mại điện tử, gán nhãn **theo khía cạnh** (aspect-based) với 10 khía cạnh và 3 cực cảm xúc.
- Bài toán khó hơn nhưng gần gũi thực tế (phù hợp cho sinh viên muốn thử thách).

Kích thước: tất cả đều dưới 20 MB, phù hợp với Colab miễn phí.

Yêu cầu bài toán (với lựa chọn 1 – UIT-VSFC):

1. Load dataset trực tiếp từ Hugging Face Hub (không cần tải thủ công).
2. Sử dụng `AutoTokenizer` và `AutoModelForSequenceClassification` với model `vinai/phobert-base` (dành riêng cho tiếng Việt). **Lưu ý:** PhoBERT yêu cầu văn bản đã được **tách từ** (word-segmented) bằng `pyvi` hoặc `VnCoreNLP`.
3. Fine-tune nhãn `sentiment` trong 2–3 epochs với `learning_rate = 2e-5`, `batch_size = 16` hoặc 32.
4. Báo cáo: accuracy, F1-score (macro), confusion matrix.
5. **Phân tích:** đưa ra 5 ví dụ mô hình dự đoán *sai* và giải thích vì sao mô hình nhầm (nhãn chồng lấn? câu quá ngắn? tiếng lóng?).
6. **Mở rộng (tùy chọn):** làm thêm bài phân loại `topic` (4 lớp) và so sánh độ khó giữa hai tác vụ.

Khung code gợi ý (UIT-VSFC với PhoBERT):

```
!pip install pyvi transformers datasets -q

from datasets import load_dataset
from pyvi import ViTokenizer
```

```

from transformers import AutoTokenizer, AutoModelForSequenceClassification,
↪ \
                                Trainer, TrainingArguments

# 1. Load dataset trực tiếp từ Hugging Face
ds = load_dataset("uitnlp/vietnamese_students_feedback")
print(ds)
# DatasetDict({
#   train: Dataset({features: ['sentence', 'sentiment', 'topic'], num_rows:
↪   11426})
#   validation: ... num_rows: 1583
#   test: ... num_rows: 3166
# })

# 2. Tách từ cho PhoBERT (yêu cầu bắt buộc!)
def segment(example):
    example["sentence"] = ViTokenizer.tokenize(example["sentence"])
    return example
ds = ds.map(segment)

# 3. Đổi tên cột label để Trainer tự nhận (chọn nhãn sentiment)
ds = ds.rename_column("sentiment", "labels")

# 4. Tokenize bằng PhoBERT
tokenizer = AutoTokenizer.from_pretrained("vinai/phobert-base")
def tokenize(batch):
    return tokenizer(batch["sentence"], truncation=True,
                      padding="max_length", max_length=128)
ds_tok = ds.map(tokenize, batched=True)

# 5. Mô hình (3 lớp: negative, neutral, positive)
model = AutoModelForSequenceClassification.from_pretrained(
    "vinai/phobert-base", num_labels=3)

# 6. Cấu hình huấn luyện
args = TrainingArguments(
    output_dir="out", num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5, weight_decay=0.01,
    eval_strategy="epoch", save_strategy="no",

```



```

fp16=True, logging_steps=50)

trainer = Trainer(model=model, args=args,
                  train_dataset=ds_tok["train"],
                  eval_dataset=ds_tok["validation"])
trainer.train()

# 7. Đánh giá trên tập test
results = trainer.evaluate(ds_tok["test"])
print(results)

```

Mẹo debug khi dùng PhoBERT

- Nếu gặp lỗi CUDA out of memory, giảm batch_size xuống 8 và bật gradient_accumulation_steps=2.
- Thay vinai/phobert-base bằng vinai/phobert-base-v2 để có kết quả tốt hơn (phiên bản cập nhật năm 2023).
- Để so sánh, có thể thử thêm model **XLM-RoBERTa** (xlm-roberta-base) – mô hình đa ngôn ngữ, không cần tách từ trước.

2.2. Bài toán 2: Phân loại chủ đề tin tức tiếng Việt

Thông tin dữ liệu – các nguồn đã kiểm chứng

Lựa chọn 1 (khuyến nghị) – UIT-VSFC topic classification:

https://huggingface.co/datasets/uitnlp/vietnamese_students_feedback

- Cùng bộ dữ liệu với Bài toán 1, nhưng dùng cột topic thay vì sentiment.
- **4 lớp chủ đề:** 0 – lecturer (giảng viên), 1 – training_program (chương trình đào tạo), 2 – facility (cơ sở vật chất), 3 – others.
- 16.175 câu, đã chia sẵn train/val/test.

Lựa chọn 2 – Vietnamese News Dataset:

<https://huggingface.co/datasets/vanhai123/vietnamese-news-dataset>

- 858 đoạn tin tức/tiêu đề tiếng Việt, gán nhãn các nhóm chủ đề như: *giáo dục, thể thao, kinh doanh, giải trí, công nghệ, ...*
- File CSV 2 cột: content, label. Cần tự chia train/val/test.

Yêu cầu:

1. Load dataset bằng `load_dataset()` từ Hugging Face.
2. Fine-tune `vinai/phobert-base` (nhớ tách từ bằng `pyvi` trước) hoặc `xlm-roberta-base` (không cần tách từ).
3. So sánh **hai chiến lược**:
 - **Feature extraction**: đóng băng toàn bộ backbone, chỉ train classifier head.
 - **Fine-tuning**: mở khóa toàn bộ, train với LR nhỏ (2×10^{-5}).
4. Báo cáo: thời gian train, accuracy trên test set, số tham số được cập nhật, F1-score macro.
5. Vẽ biểu đồ loss/accuracy theo epoch, confusion matrix cho 4 lớp.

```
from datasets import load_dataset
ds = load_dataset("uitnlp/vietnamese_students_feedback")
ds = ds.rename_column("topic", "labels")    # Dùng nhãn topic (4 lớp)
# ... (tiếp tục giống Bài toán 1, chỉ đổi num_labels=4)
```

2.3. Bài toán 3: Nhận dạng thực thể có tên (NER) tiếng Việt – COVID-19

Thông tin dữ liệu – các nguồn đã kiểm chứng

Dataset: PhoNER_COVID19 – bộ NER tiếng Việt đầu tiên về đại dịch COVID-19, từ VinAI Research (NAACL 2021).

Lựa chọn 1 (khuyến nghị) – GitHub chính thức của VinAI:

https://github.com/VinAIRResearch/PhoNER_COVID19

- 10.027 câu, 34.984 thực thể, chia sẵn train (5.027) / dev (2.000) / test (3.000).
- **10 loại thực thể:** PATIENT_ID, PERSON_NAME, AGE, GENDER, OCCUPATION, LOCATION, ORGANIZATION, SYMPTOM_AND_DISEASE, TRANSPORTATION, DATE.
- Dữ liệu dạng **CoNLL format** (mỗi dòng: token \t nhãn BIO).
- Có sẵn cả hai phiên bản: word-level và syllable-level.

Lựa chọn 2 – Hugging Face mirror:

https://huggingface.co/datasets/SEACrowd/pho_ner_covid

- Cần cài thêm: `pip install seacrowd`

- Load: `load_dataset("SEACrowd/pho_ner_covid", trust_remote_code=True)`

Yêu cầu:

1. Clone repo GitHub hoặc dùng HF mirror để tải dữ liệu.
2. Dùng `AutoModelForTokenClassification` với `vinai/phobert-base` (đã được chứng minh tốt nhất cho dataset này).
3. Xử lý **alignment** giữa word-level labels và subword tokens của PhoBERT (dùng `word_ids()`).
4. Đánh giá bằng `seqeval`: precision, recall, F1 **theo thực thể** (không phải theo token).
5. **Demo**: nhập 3 câu tin tức COVID-19 mới, trích xuất và highlight các entity.

```
# Tải dữ liệu PhoNER_COVID19 từ GitHub
!git clone https://github.com/VinAIRsearch/PhoNER_COVID19.git

# Đọc file CoNLL (dùng word-level)
def read_conll(path):
    sents, tags, cur_s, cur_t = [], [], [], []
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                if cur_s:
                    sents.append(cur_s); tags.append(cur_t)
                    cur_s, cur_t = [], []
            else:
                tok, tag = line.split("\t")
                cur_s.append(tok); cur_t.append(tag)
        if cur_s:
            sents.append(cur_s); tags.append(cur_t)
    return sents, tags

train_toks, train_tags = read_conll(
    "PhoNER_COVID19/data/word/train_word.conll")

# Alignment với subword tokens của PhoBERT
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("vinai/phobert-base")
```

```
def tokenize_and_align(tokens_list, tags_list, label2id):
    encoded = tokenizer(tokens_list, is_split_into_words=True,
                        truncation=True, padding="max_length",
                        max_length=256, return_tensors=None)

    labels = []
    for i, tags in enumerate(tags_list):
        word_ids = encoded.word_ids(batch_index=i)
        lab = [-100 if wid is None else label2id[tags[wid]]
               for wid in word_ids]
        labels.append(lab)
    encoded["labels"] = labels
    return encoded

# Đánh giá bằng seqeval
!pip install seqeval -q
from seqeval.metrics import classification_report, f1_score
# ... tính F1 trên tập test
```

2.4. Bài toán 4: Hỏi đáp trích xuất (Extractive QA) tiếng Việt

Thông tin dữ liệu – các nguồn đã kiểm chứng

Dataset: UIT-ViQuAD 2.0 – bộ QA tiếng Việt chuẩn, tương đương SQuAD cho tiếng Anh.

Nguồn chính trên Hugging Face:

<https://huggingface.co/datasets/taidng/UIT-ViQuAD2.0>

- Hơn **23.000 cặp câu hỏi – trả lời** được người gán nhãn thủ công, dựa trên 174 bài viết từ Wikipedia tiếng Việt.
- Phiên bản 2.0 bổ sung hơn **12.000 câu hỏi không thể trả lời** (unanswerable questions) với trường `is_impossible`.
- Định dạng giống hệt SQuAD $\Rightarrow$ có thể tái sử dụng nhiều code SQuAD.
- Load: `load_dataset("taidng/UIT-ViQuAD2.0")`

Yêu cầu:

1. Load dataset, khám phá cấu trúc: `context`, `question`, `answers` (list các span), `is_impossible`.
2. Chỉ lấy **3.000–5.000** mẫu cho train để chạy nhanh trên Colab.

3. Fine-tune `xlm-roberta-base` hoặc `nguyenvulebinh/vi-mrc-base` (đã pretrained cho QA tiếng Việt).
4. Đánh giá: **Exact Match (EM)** và **F1-score** trên dev set.
5. **Demo:** viết hàm `answer_question(context, question)` và test với 3 câu hỏi tự soạn trên 1 đoạn văn Wikipedia tiếng Việt.

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForQuestionAnswering,
    pipeline

ds = load_dataset("taidng/UIT-ViQuAD2.0")
print(ds["train"][0]) # Xem 1 ví dụ

# Dùng model đã pretrained cho QA tiếng Việt
qa_pipe = pipeline("question-answering",
                    model="nguyenvulebinh/vi-mrc-base",
                    tokenizer="nguyenvulebinh/vi-mrc-base")

context = """Đại học Khoa học Tự nhiên, Đại học Quốc gia TP.HCM
là trường đại học đào tạo, nghiên cứu khoa học cơ bản..."""
question = "Trường ĐHKHTN thuộc đại học nào?"
print(qa_pipe(question=question, context=context))
```

2.5. Bài toán 5: Tóm tắt văn bản tiếng Việt (Abstractive Summarization)

Thông tin dữ liệu – các nguồn đã kiểm chứng

Dataset: VietNews – tóm tắt bài báo tiếng Việt.

Nguồn chính trên Hugging Face:

<https://huggingface.co/datasets/harouzie/vietnews>

- 144.000 cặp (bài báo, tóm tắt) tiếng Việt từ các báo lớn, có sẵn chia train/val/test (99.1k / 22.2k / 22.5k).
- Đã auto-convert sang Parquet, load trực tiếp bằng `load_dataset()`.
- Kích thước file: 247 MB.

Nguồn thay thế – VNDS trên GitHub:

<https://github.com/ThanhChinhBK/vietnews>

- Bản gốc bài báo VNDS (Nguyen et al., 2019), định dạng `.txt`.

Yêu cầu:

1. Load dataset, **chỉ lấy 3.000–5.000** mẫu cho train để chạy nhanh (bài toán seq2seq tốn nhiều thời gian hơn classification).
2. Fine-tune một trong các model sau:
 - VietAI/vit5-base-vietnews-summarization (đã pretrained cho bài này – chỉ cần fine-tune nhẹ).
 - vinai/bartpho-syllable – BARTpho của VinAI, dành cho tóm tắt tiếng Việt.
3. Đánh giá bằng **ROUGE-1, ROUGE-2, ROUGE-L** (dùng thư viện rouge-score).
4. **Phân tích định tính:** chọn 5 bài báo, so sánh tóm tắt do mô hình sinh ra với tóm tắt tham chiếu của con người. Nhận xét: mô hình có xu hướng lặp từ không? Có thông tin sai lệch (hallucination) không?

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, \
    Seq2SeqTrainer, Seq2SeqTrainingArguments, \
    DataCollatorForSeq2Seq

ds = load_dataset("harouzie/vietnews")
ds["train"] = ds["train"].select(range(3000)) # Lấy 3k mẫu

# Dùng ViT5 đã pretrained cho tóm tắt
checkpoint = "VietAI/vit5-base-vietnews-summarization"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)

def preprocess(batch):
    inputs = ["vietnews: " + doc for doc in batch["article"]]
    model_inputs = tokenizer(inputs, max_length=512, truncation=True)
    labels = tokenizer(text_target=batch["abstract"],
                       max_length=128, truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

ds_tok = ds.map(preprocess, batched=True,
                remove_columns=ds["train"].column_names)

args = Seq2SeqTrainingArguments(
    output_dir="out", num_train_epochs=2,
    per_device_train_batch_size=4,
```

```

per_device_eval_batch_size=8,
gradient_accumulation_steps=2,
learning_rate=3e-5, weight_decay=0.01,
predict_with_generate=True, fp16=True,
eval_strategy="epoch", save_strategy="no")

trainer = Seq2SeqTrainer(
    model=model, args=args,
    train_dataset=ds_tok["train"],
    eval_dataset=ds_tok["validation"].select(range(200)),
    tokenizer=tokenizer,
    data_collator=DataCollatorForSeq2Seq(tokenizer, model=model))
trainer.train()

# Tính ROUGE
!pip install rouge-score -q
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
                                    use_stemmer=False)

```

Phân mở rộng: Transformer cho dữ liệu ngoài ngôn ngữ

Ý tưởng chung

Transformer **không** phải chỉ dành cho ngôn ngữ. Bất kỳ dữ liệu nào có thể biểu diễn dưới dạng **chuỗi token** (sequence of tokens) hoặc **tập các vector đặc trưng** đều có thể dùng Transformer. Ý tưởng chung:

- **Time series / tín hiệu 1D:** mỗi bước thời gian là một "token".
- **Hình ảnh:** chia ảnh thành các patch 16×16 , mỗi patch là một token (Vision Transformer – ViT).
- **Âm thanh:** dùng spectrogram (Mel-spectrogram) rồi chia patch như ảnh, hoặc dùng wav2vec/whisper.
- **Tabular/đồ thị:** mỗi hàng/nút là một token, thêm encoding đặc biệt.

Sinh viên chọn **1 bài toán** trong 4 bài dưới đây như là **giải pháp thay thế** cho 5 bài NLP ở trên.

2.6. Bài toán 6 (ngoài NLP): Dự báo chuỗi thời gian – Doanh số bán hàng

Thông tin dữ liệu

Dataset: Store Sales - Time Series Forecasting (Kaggle Competition).

Link Kaggle:

<https://www.kaggle.com/competitions/store-sales-time-series-forecasting>

- Dữ liệu doanh số từ **chuỗi siêu thị Corporación Favorita** ở Ecuador, từ 01/2013 đến 08/2017.
- 54 cửa hàng $\times$ 33 nhóm sản phẩm = 1.782 chuỗi thời gian.
- **Nhiệm vụ:** dự báo doanh số 16 ngày tiếp theo.
- Kèm nhiều **biến ngoại sinh**: giá dầu, ngày lễ, khuyến mại, trận động đất 2016.
- Tổng dữ liệu ~ 120 MB.

Tải bằng Kaggle API trên Colab:

```
!pip install kaggle -q
# Upload file kaggle.json (tải từ https://www.kaggle.com/settings)
from google.colab import files
files.upload() # kaggle.json
!mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle competitions download -c store-sales-time-series-forecasting
!unzip store-sales-time-series-forecasting.zip -d store_sales/
```

Yêu cầu bài toán:

1. **Tiền xử lý:** chọn 1 cửa hàng + 1 nhóm sản phẩm (ví dụ `store_nbr=1`, `family=BEVERAGES`) để đơn giản hóa. Điều này cho ta **1 chuỗi thời gian đơn lẻ** dài ~ 1.680 ngày.
2. **Tạo window:** cửa sổ đầu vào 30 ngày, cửa sổ đầu ra 7 ngày (ngắn hạn). Chia train/val theo thời gian (không shuffle ngẫu nhiên!).
3. Xây dựng **Transformer Encoder** với:
 - `input_projection = nn.Linear(n_features, d_model)` để chiếu tín hiệu vào không gian d_{model} .
 - `PositionalEncoding` như đã cài đặt ở Phần 1.
 - 2-3 lớp `TransformerEncoderLayer` của PyTorch hoặc tự cài.

- `output_projection = nn.Linear(d_model, 1)` cho mỗi bước dự báo.
4. Loss: **MSE** hoặc **SMAPE**. So sánh với baseline đơn giản (naive forecast: dự báo bằng giá trị ngày trước).
 5. **Demo**: vẽ biểu đồ **dự báo vs. thực tế** cho 7 ngày cuối tập validation.

```
import torch
import torch.nn as nn
import pandas as pd
import numpy as np

# Đọc và lọc dữ liệu
df = pd.read_csv("store_sales/train.csv", parse_dates=["date"])
df = df[(df["store_nbr"] == 1) & (df["family"] == "BEVERAGES")]
df = df.sort_values("date").reset_index(drop=True)
sales = df["sales"].values  # Chuỗi 1D

# Chuẩn hóa
mean, std = sales.mean(), sales.std()
sales_norm = (sales - mean) / std

# Tạo window (30 -> 7)
def make_windows(arr, enc_len=30, dec_len=7):
    X, Y = [], []
    for i in range(len(arr) - enc_len - dec_len):
        X.append(arr[i : i+enc_len])
        Y.append(arr[i+enc_len : i+enc_len+dec_len])
    return np.array(X), np.array(Y)

X, Y = make_windows(sales_norm)  # X: [N, 30], Y: [N, 7]
# Chia theo thời gian: 80% đầu cho train, 20% sau cho val
split = int(0.8 * len(X))
X_train, Y_train = X[:split], Y[:split]
X_val, Y_val = X[split:], Y[split:]

class TimeSeriesTransformer(nn.Module):
    def __init__(self, d_model=64, n_head=4, n_layers=2, dec_len=7):
        super().__init__()
        self.input_proj = nn.Linear(1, d_model)  # Mỗi bước là scalar
        self.pos_enc = nn.Embedding(100, d_model)  # Learnable PE
        enc_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=n_head,
```

```

        dim_feedforward=128, batch_first=True, dropout=0.1)
    self.encoder = nn.TransformerEncoder(enc_layer, num_layers=n_layers)
    self.head = nn.Linear(d_model, dec_len)  # Dự báo dec_len bước

    def forward(self, x):
        # x: [B, 30]
        x = x.unsqueeze(-1)  # [B, 30, 1]
        h = self.input_proj(x)  # [B, 30, d_model]
        pos = torch.arange(h.size(1), device=h.device)
        h = h + self.pos_enc(pos)  # Add PE
        h = self.encoder(h)  # [B, 30, d_model]
        return self.head(h[:, -1, :])  # Lấy token cuối [B,
        ↪ 7]

model = TimeSeriesTransformer().cuda()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()
# ... vòng lặp training tương tự bài NLP

```

Gợi ý mở rộng

- Thêm các **biến phụ** (giá dầu, ngày lễ) vào input: `input_proj = nn.Linear(n_features, d_model)` với `n_features > 1`.
- Thử mô hình **Informer** hoặc **Temporal Fusion Transformer (TFT)** trong thư viện `pytorch-forecasting`:
<https://pytorch-forecasting.readthedocs.io/en/stable/tutorials/stallion.html>

2.7. Bài toán 7 (ngoài NLP): Phân loại nhịp tim ECG bằng Transformer

Thông tin dữ liệu

Dataset: ECG Heartbeat Categorization Dataset.

Link Kaggle:

<https://www.kaggle.com/datasets/shayanfazeli/heartbeat>

- **MIT-BIH Arrhythmia Database:** khoảng 109.000 nhịp tim đã tiền xử lý, chia 5 lớp theo chuẩn AAMI:
 - **N** – normal beat (bình thường)

- **S** – supraventricular ectopic
 - **V** – ventricular ectopic
 - **F** – fusion beat
 - **Q** – unknown beat
- Mỗi mẫu là **chuỗi 187 điểm** (đã resample), giá trị đã chuẩn hóa về $[0, 1]$.
 - File CSV: `mitbih_train.csv` (87.554 mẫu) + `mitbih_test.csv` (21.892 mẫu).
Cột 0–186 là tín hiệu, cột 187 là nhãn 0-4.
 - **Nhẹ** (~100 MB), không cần Kaggle GPU.

Yêu cầu bài toán:

1. Tải dataset, load bằng `pandas`. Quan sát sự **mất cân bằng lớp** (class N chiếm ~83%).
2. Xây Transformer Encoder cho bài **phân loại chuỗi 1D**:
 - Coi mỗi điểm (scalar) hoặc nhóm k điểm (patch) là 1 token.
 - Thêm `[CLS]` token đầu chuỗi (learnable), và Positional Encoding.
 - 2-4 lớp Encoder với `d_model=64`, `n_head=4`.
 - Lấy biểu diễn của `[CLS]` để phân loại qua MLP.
3. Loss: **CrossEntropyLoss** với `class_weight` (để xử lý imbalance).
4. Đánh giá: **Accuracy**, **F1-score macro**, **confusion matrix**. Chú ý: accuracy không đủ vì imbalance!
5. So sánh với baseline CNN-1D đơn giản (2 lớp Conv1D + MaxPool + FC).

```
import pandas as pd, torch, torch.nn as nn
train = pd.read_csv("mitbih_train.csv", header=None)
test  = pd.read_csv("mitbih_test.csv",  header=None)

X_train = train.iloc[:, :187].values.astype("float32") # [N, 187]
y_train = train.iloc[:, 187].values.astype("int64")
X_test  = test.iloc[:, :187].values.astype("float32")
y_test  = test.iloc[:, 187].values.astype("int64")

class ECGTransformer(nn.Module):
    def __init__(self, seq_len=187, d_model=64, n_head=4,
                  n_layers=3, n_classes=5):
        super().__init__()
```

```

self.input_proj = nn.Linear(1, d_model)
self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
self.pos_enc = nn.Embedding(seq_len + 1, d_model)
enc = nn.TransformerEncoderLayer(d_model, n_head,
                                dim_feedforward=128,
                                batch_first=True, dropout=0.1)
self.encoder = nn.TransformerEncoder(enc, n_layers)
self.classifier = nn.Sequential(
    nn.Linear(d_model, 32), nn.ReLU(),
    nn.Linear(32, n_classes))

def forward(self, x):
    # x: [B, 187]
    B = x.size(0)
    x = x.unsqueeze(-1) # [B, 187, 1]
    h = self.input_proj(x) # [B, 187, d]
    cls = self.cls_token.expand(B, -1, -1) # [B, 1, d]
    h = torch.cat([cls, h], dim=1) # [B, 188, d]
    pos = torch.arange(h.size(1), device=h.device)
    h = h + self.pos_enc(pos)
    h = self.encoder(h) # [B, 188, d]
    return self.classifier(h[:, 0]) # CLS token

# Xử lý class imbalance
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
cw = compute_class_weight("balanced", classes=np.arange(5), y=y_train)
loss_fn = nn.CrossEntropyLoss(weight=torch.tensor(cw,
    ↪ dtype=torch.float32).cuda())

```

2.8. Bài toán 8 (ngoài NLP): Vision Transformer (ViT) cho ảnh CIFAR-10

Thông tin dữ liệu

Dataset: CIFAR-10 – 60.000 ảnh 32×32 RGB, 10 lớp.

Có sẵn trong torchvision:

Link Kaggle (nếu cần): <https://www.kaggle.com/c/cifar-10>

- 10 lớp: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- 50.000 train + 10.000 test. **Nhẹ** (~ 170 MB, đã tải sẵn qua torchvision).

```
from torchvision.datasets import CIFAR10
trainset = CIFAR10(root='./data', train=True, download=True)
testset = CIFAR10(root='./data', train=False, download=True)
```

Ý tưởng Vision Transformer (ViT): Chia ảnh 32×32 thành các **patch** $4 \times 4 = 16$ pixel, cho ra $8 \times 8 = 64$ patch. Mỗi patch flatten thành vector $16 \times 3 = 48$ chiều rồi chiếu vào d_{model} . Thêm [CLS] token và positional embedding.

Yêu cầu bài toán:

1. **Chuẩn bị:** resize ảnh (nếu muốn), normalize theo mean/std của CIFAR.
2. Cài đặt một ViT **nhỏ**:
 - `patch_size=4`, `d_model=128`, `n_head=4`, `n_layers=4`.
 - Tổng $\sim 1\text{M}$ tham số, chạy được trên Colab T4.
3. So sánh 2 chiến lược:
 - **Huấn luyện từ đầu** (from scratch) trên CIFAR-10.
 - **Fine-tune** từ ViT pretrained: google/vit-base-patch16-224-in21k qua transformers.
4. Báo cáo accuracy trên tập test. Dự đoán: train from scratch chỉ đạt $\sim 60\text{-}70\%$, fine-tune đạt $>95\%$.
5. Giải thích: tại sao ViT từ đầu **kém** CNN trên dataset nhỏ như CIFAR-10? (Gợi ý: thiếu *inductive bias* về cấu trúc không gian; cần nhiều data hơn).

```
import torch, torch.nn as nn

class SmallViT(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_channels=3,
                  d_model=128, n_head=4, n_layers=4, n_classes=10):
        super().__init__()
        self.patch_size = patch_size
        n_patches = (img_size // patch_size) ** 2  # 64
```

```

# Patch embedding qua Conv2d kernel=patch_size, stride=patch_size
self.patch_emb = nn.Conv2d(in_channels, d_model,
                             kernel_size=patch_size,
                             ↪ stride=patch_size)

self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
self.pos_emb = nn.Parameter(torch.zeros(1, n_patches + 1,
    ↪ d_model))

enc = nn.TransformerEncoderLayer(d_model, n_head,
                                dim_feedforward=4*d_model,
                                batch_first=True, dropout=0.1)

self.encoder = nn.TransformerEncoder(enc, n_layers)
self.head = nn.Linear(d_model, n_classes)

def forward(self, x):
    # x: [B, 3, 32, 32]
    B = x.size(0)
    h = self.patch_emb(x) # [B, d, 8, 8]
    h = h.flatten(2).transpose(1, 2) # [B, 64, d]
    cls = self.cls_token.expand(B, -1, -1)
    h = torch.cat([cls, h], dim=1) # [B, 65, d]
    h = h + self.pos_emb
    h = self.encoder(h)
    return self.head(h[:, 0]) # CLS

# Hoặc dùng pretrained ViT từ Hugging Face
from transformers import ViTForImageClassification, ViTImageProcessor
model = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224-in21k", num_labels=10)
processor = ViTImageProcessor.from_pretrained(
    "google/vit-base-patch16-224-in21k")

```

2.9. Bài toán 9 (ngoài NLP): Phân loại lệnh giọng nói (Audio Classification)

Thông tin dữ liệu

Dataset: Speech Commands (Google) – 35 lệnh giọng nói tiếng Anh ngắn.

Có sẵn trên Hugging Face:

Hoặc dùng version nhỏ trên Kaggle:

<https://www.kaggle.com/datasets/neeakurelli/google-speech-commands>

- Hơn 100.000 clip WAV **1 giây**, 16 kHz, mỗi clip là một từ tiếng Anh (*yes, no, up, down, left, right, stop, go, ...*).
- **Chỉ lấy tập con 10 lớp phổ biến để giảm độ khó.**

```
from datasets import load_dataset
ds = load_dataset("speech_commands", "v0.02", split="train[:10%]")
```

Yêu cầu bài toán:

1. **Tiền xử lý:** chuyển audio 1D thành Mel-spectrogram 2D bằng `torchaudio.transforms.MelSpec`
2. Dùng mô hình **AST (Audio Spectrogram Transformer)** pretrained:
 - Load: MIT/ast-finetuned-audioset-10-10-0.4593
 - Hoặc nhẹ hơn: facebook/wav2vec2-base (xử lý audio raw).
3. Fine-tune mô hình trên 10 lệnh, 2–3 epoch.
4. Đánh giá: accuracy, confusion matrix.
5. **Demo:** ghi âm 1 giây (dùng `IPython.display.Audio` trên Colab) và cho mô hình dự đoán.

```
from datasets import load_dataset, Audio
from transformers import AutoFeatureExtractor, \
    AutoModelForAudioClassification, \
    Trainer, TrainingArguments

# Chọn 10 lớp phổ biến
keep_labels = ["yes", "no", "up", "down", "left", "right", "stop", "go", "on", "off"]
ds = load_dataset("speech_commands", "v0.02")
ds = ds.filter(lambda ex: ex["label"] < len(keep_labels))
ds = ds.cast_column("audio", Audio(sampling_rate=16000))
```

```
# Feature extractor cho AST
checkpoint = "MIT/ast-finetuned-audioset-10-10-0.4593"
feat_ext = AutoFeatureExtractor.from_pretrained(checkpoint)

def preprocess(batch):
    audio = batch["audio"]["array"]
    inputs = feat_ext(audio, sampling_rate=16000, return_tensors="pt")
    batch["input_values"] = inputs.input_values[0]
    batch["labels"] = batch["label"]
    return batch

ds_tok = ds.map(preprocess, remove_columns=ds["train"].column_names)
model = AutoModelForAudioClassification.from_pretrained(
    checkpoint, num_labels=10, ignore_mismatched_sizes=True)
# ... fine-tune như các bài NLP
```

So sánh 4 bài toán phi-NLP

Bài	Lĩnh vực	Loại dữ liệu	Độ khó	Thời gian
6	Time series	Chuỗi 1D liên tục	Trung bình	30-60p
7	Y tế / ECG	Chuỗi 1D phân loại	Dễ	30-45p
8	Thị giác máy tính	Ảnh 2D	Trung bình	45-90p
9	Xử lý tín hiệu âm thanh	Audio / spectrogram	Khó	60-90p

Gợi ý chọn bài:

- Sinh viên **Toán - Tin, Khoa học dữ liệu**: nên chọn Bài 6 (time series, ứng dụng tài chính/kinh tế) hoặc Bài 7 (y sinh, ứng dụng thực tế).
- Sinh viên thích **computer vision**: chọn Bài 8 (ViT).
- Sinh viên thích **speech & audio**: chọn Bài 9.

Tiết kiệm tài nguyên khi chạy Phần 2

- Bật **mixed precision** (fp16=True trong TrainingArguments) để giảm 50% bộ nhớ GPU và tăng tốc.
- Dùng max_length=128 hoặc 256 thay vì 512 khi có thể.
- Giảm batch_size xuống 4-8 nếu gặp CUDA out of memory, kết hợp gradient_accumulation_steps=2--4.

- Lưu checkpoint lên Google Drive để phòng Colab ngắt kết nối.
- Với bài **summarization** (seq2seq): dùng model **base** thay vì **large**, giảm `max_target_length` xuống 64–128.

2.10. Tiêu chí chấm điểm Phần 2

Tiêu chí	Điểm
Code chạy được, kết quả tái lập (set seed, log đầy đủ)	2.0
Tiền xử lý dữ liệu đúng (tokenize, split, align nhãn)	1.5
Huấn luyện & lựa chọn hyperparameter hợp lý	2.0
Đánh giá đầy đủ (metric phù hợp, biểu đồ, confusion matrix/ROUGE)	2.0
Phân tích sai sót, thảo luận hướng cải tiến	1.5
Trình bày báo cáo rõ ràng, chú thích tiếng Việt	1.0
Tổng	10.0

Tài liệu tham khảo khuyến nghị

Về Transformer:

- Vaswani et al., *"Attention Is All You Need"*, NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- Jay Alammar, *"The Illustrated Transformer"*. <https://jalammar.github.io/illustrated-transformer/>
- Hugging Face Course (miễn phí): <https://huggingface.co/learn/nlp-course>
- PyTorch tutorial chính thức: https://pytorch.org/tutorials/beginner/transformer_tutorial.html

Về mô hình và dữ liệu tiếng Việt:

- Nguyen & Nguyen, *"PhoBERT: Pre-trained language models for Vietnamese"*, EMNLP 2020.
- Tran et al., *"BARTpho: Pre-trained Sequence-to-Sequence Models for Vietnamese"*, Interspeech 2022.

- Phan et al., "*ViT5: Pretrained Text-to-Text Transformer for Vietnamese Language Generation*", NAACL 2022.
- Truong et al., "*COVID-19 Named Entity Recognition for Vietnamese*", NAACL 2021. https://github.com/VinAIRResearch/PhoNER_COVID19
- Nguyen et al., "*A Vietnamese Dataset for Evaluating Machine Reading Comprehension*", COLING 2020.
- **UIT NLP Group datasets:** <https://sites.google.com/uit.edu.vn/uit-nlp/datasets>
- **Awesome Vietnamese NLP** (danh sách tài nguyên tiếng Việt): <https://github.com/vndee/awesome-vietnamese-nlp>

— Chúc các em học tốt! —